Introduction to Using HPRC Computing Clusters

Presenter: Dr. Lujun Zhai
Host Institution: Prairie View A&M University
Workshop Venue: University of Houston
Aug 8, 2024

Agenda

- Introduction to computing systems platform (ACCESS)
- Introduction to High Performance Research Computing (HPRC)
- Performance comparison of Advanced GPUs and IPUs
- Hands on AI training with A30 GPUs .

Introduction to ACCESS

- ACCESS (Advanced CyberInfrastructure for Computational and Emerging Sciences) is a program funded by the NSF, designed to help researchers and educators with using HPC resources and cyberinfrastructure. Its aim is to make these resources more accessible to researchers. We can use the nation's advanced computing systems and services **at no cost**.

- TAMU is a member of the ACCESS program. TAMU'S High Performance Research Computing (HPRC) provides access to its high-performance computing resources, such as the ACES cluster, & Launcher cluster to researchers and educators through the ACCESS platform. HPRC provides a wide range of advanced AI accelerators.

- Collaborative connection between PVAMU with TAMU.

Introduction to HPRC

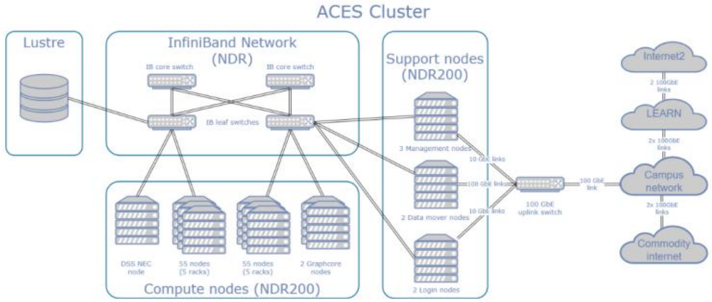
The HPRC group currently administers six HPC clusters.

- ACES
- Launch
- FASTER
- Grace
- ViDaL
- Lonestar6

Introduction to ACES

ACES: A Dell x86 HPC Cluster

System Name:	ACES
Operating System:	Red Hat Enterprise 8
Total Compute Cores/Nodes:	11,888 cores 130 nodes
Compute Nodes:	110 Intel Sapphire Rapids Nodes 17 Intel Ice Lake Nodes 1 AMD Rome Graphcore Node with 16 Mk2 Colossus GC200 IPUs 1 Intel Ice Lake Graphcore Node with 16 Bow-2000 IPUs 1 Intel Cascade Lake Node with 8 NEC Vector Engine Type 20B-P cards
Composable Components	30 NVIDIA H100 GPUs 4 NVIDIA A30 GPUs 3 Bittware Agilex FPGAs 2 Intel D5005 FPGAs 2 NextSilicon coprocessors 48 Intel Optane SSDs 120 Intel GPU Max 1100
Interconnect:	NVIDIA Mellanox NDR200 InfiniBand (MPI and storage) Liqid PCIe Gen4 Fabrics (composability) Liqid PCIe Gen5 Fabrics (composability)



HPRC's ACES Cluster

ACES is a Dell cluster with rich accelerator testbed. 130 nodes. ACES cluster has a variety of HPC resources, Graphcore Intelligent Processing Unit (IPU), NVidia A100/H100 GPUs, Intel Max GPUs, and FPGA systems.

Introduction to FASTER

FASTER: A Dell x86 HPC Cluster

System Name:	FASTER
Host Name:	faster.hprc.tamu.edu
Operating System:	Rocky Linux 8
Total Compute Cores/Nodes:	11,520 cores 180 nodes
Compute Nodes:	180 64-core compute nodes, each with 256GB RAM
Composable GPUs:	200 T4 16GB GPUs 40 A100 40GB GPUs 8 A10 24GB GPUs 4 A30 24GB GPUs 8 A40 48GB GPUs
Interconnect:	Mellanox HDR100 InfiniBand (MPI and storage) Liqid PCIe Gen4 (GPU composability)
Peak Performance:	1.2 PFLOPS

HPRC's FASTER Cluster

- FASTER is a 184-node Dell cluster .
- FASTER composable cluster allow user to add up to 32 GPUs to a single computing node.
- A100 GPUs, A10 GPUs, A30 GPUs, A40 GPUs and T4 GPUs are distributed and composable via Liqid PCIe fabrics.

Introduction to Launch

Launch: A Dell x86 HPC Cluster

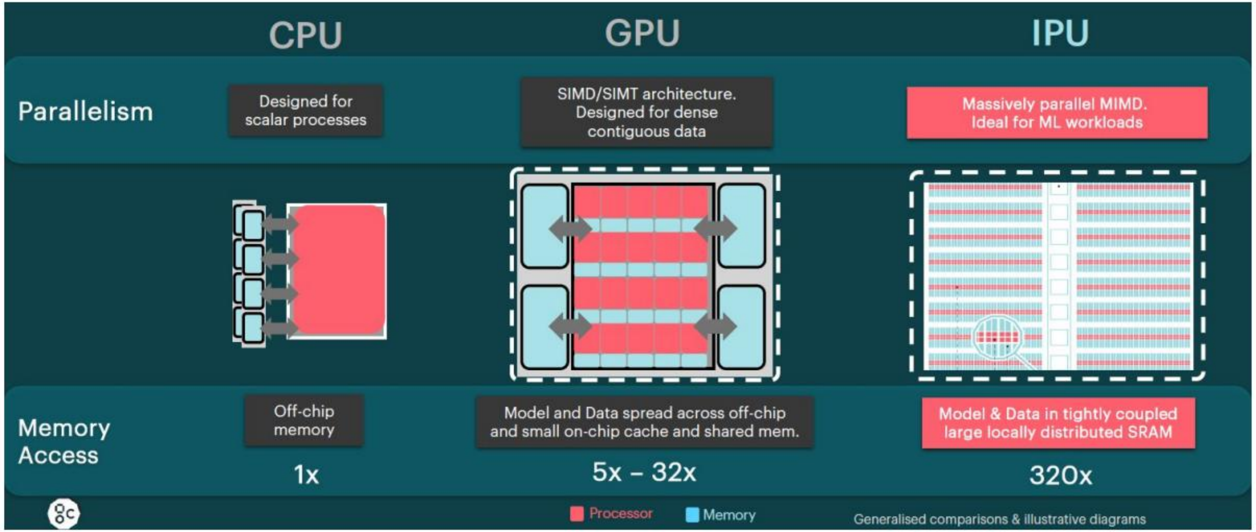
System Name:	Launch
Operating System:	Red Hat Enterprise 8
Total Compute Cores/Nodes:	8640 cores 45 nodes
Compute Nodes:	35 AMD Genoa CPU Nodes with 384 GB 10 AMD Genoa GPU nodes each with 2 NVIDIA A30s and 768 GB
Interconnect:	NVIDIA HDR100
Peak Performance:	436 TFLOPS
Global Disk:	2.2 PB (usable) via Dell storage server
File System:	NFS
Batch Facility:	Slurm by SchedMD
Location:	West Campus Data Center
Production Date:	Late 2023 or early 2024

Launch Cluster

- .Launch is a Dell Linux cluster with 45 compute nodes (8,640 cores) and 2 login nodes.
- 35 Genoa CPU Nodes.10 NVIDIA A30 GPU nodes.

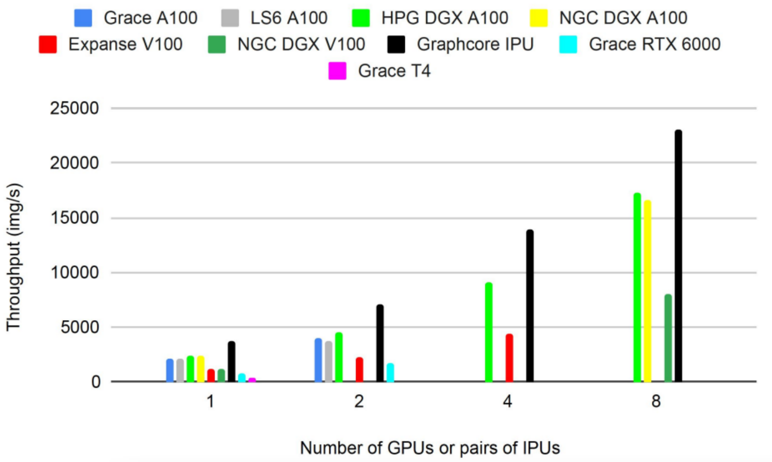
More detailed cluster information can be founded here: **https://hprc.tamu.edu/resources/**

Collaborative work on Graphcore Intelligence Processing Unit (IPU)



- Graphcore's IPU uses on-chip SRAM as a key component. This design allows for efficient and high-speed memory access.
- The IPU architecture is designed to minimize data movement between SRAM and off-chip memory, which helps reduce power consumption and latency, making it well-suited for AI and machine learning workloads.

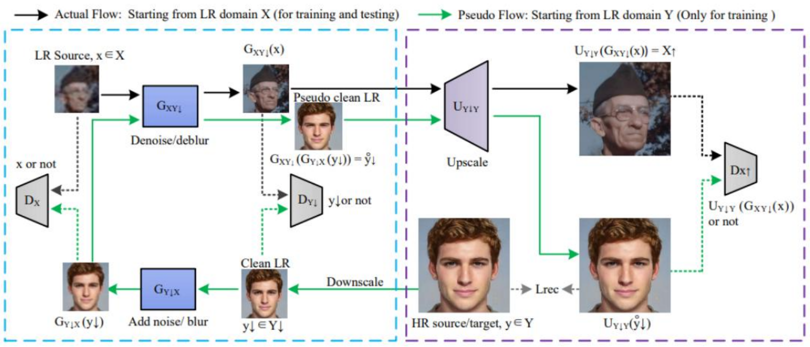
Performance of Intelligence Processing Unit (IPU)



- Comparison of GPUs and IPUs on the ResNet 50 model in PyTorch with mixed precision.
- Scaling is similar for GPUs and IPUs, with similar magnitude per-device.

- While NVIDIA A100 series GPUs are indeed powerful, when compared to the Graphcore IPU, they still fall short by a considerable margin.
- The IPU has a linear scaling performance with the amount of IPU.

User case of Intelligence Processing Unit (IPU)



a) Unpaired blur/noise correction network b) Pseudo-paired SR network

Figure: Architecture of Trans-CycleGAN for historical image super-resolution.

Table 2: Training performance of IPUs on the TransCycleGAN

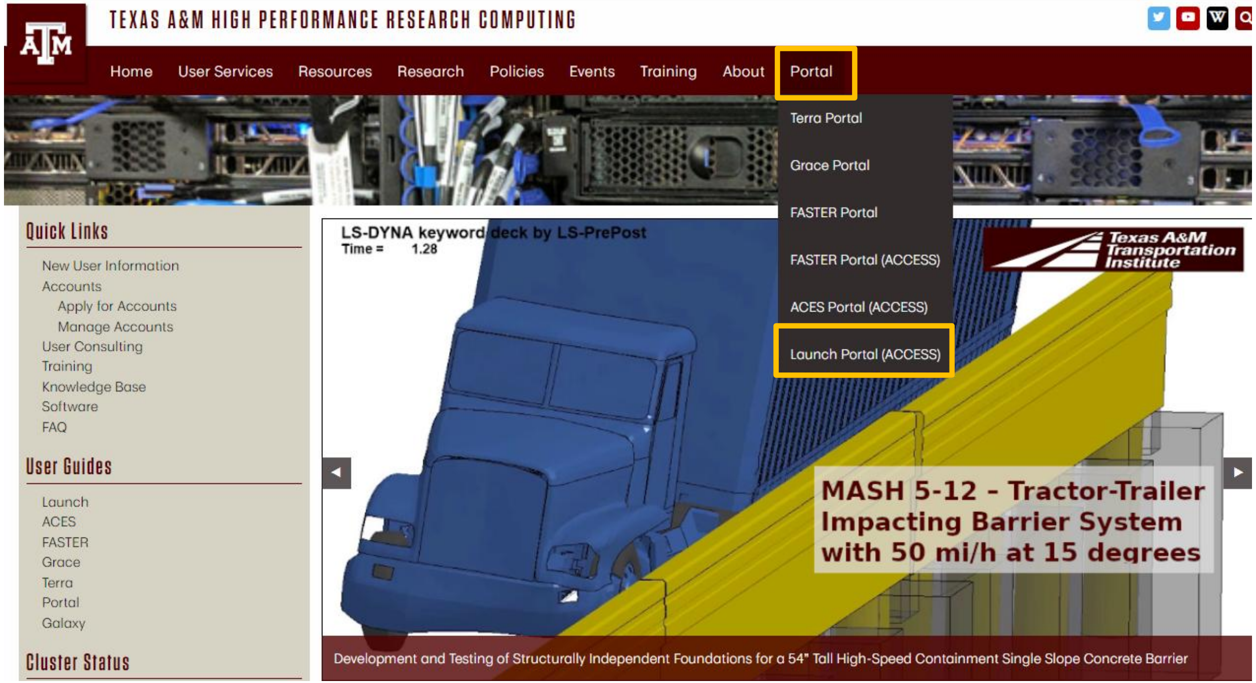
Accelerator	Count	Images/sec
IPU	1	438
IPU	16	6552

- As a use case, we ported our image super-resolution model, TransCyCycleGAN, to the Graphcore IPU.
- Joint publication in the PEARC23: Porting AI/ML Models to Intelligence Processing Units (IPUs).

Lab 1: Logging to Launch Cluster & NVIDIA A30 GPUs

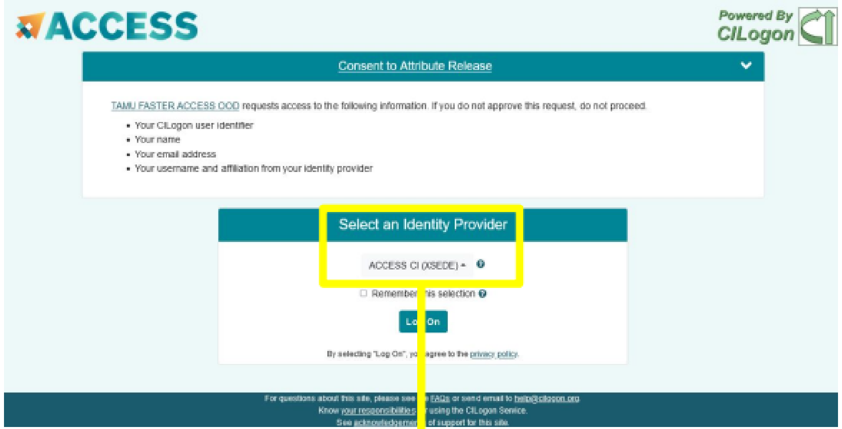
Log into Launch Using the HPRC Portal

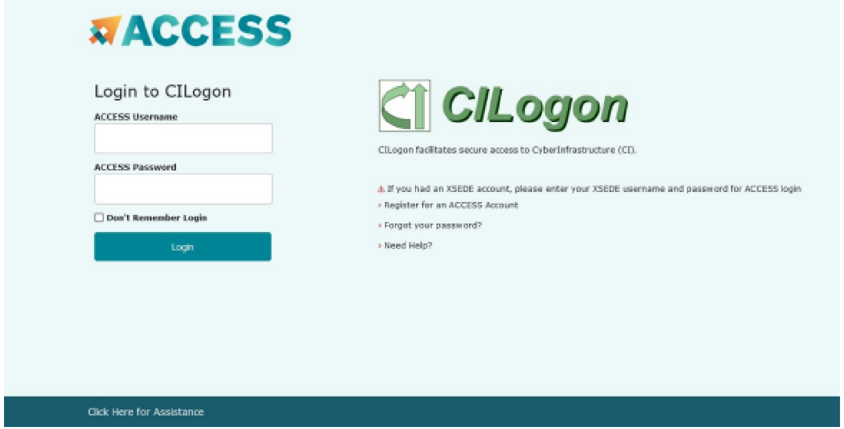
HPRC webpage: https://hprc.tamu.edu/, Portal dropdown menu

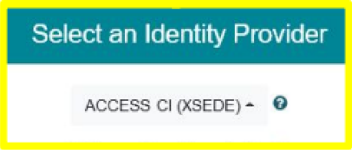


Accessing Launch via the Launch Portal (ACCESS)

Log-in using your ACCESS credentials.





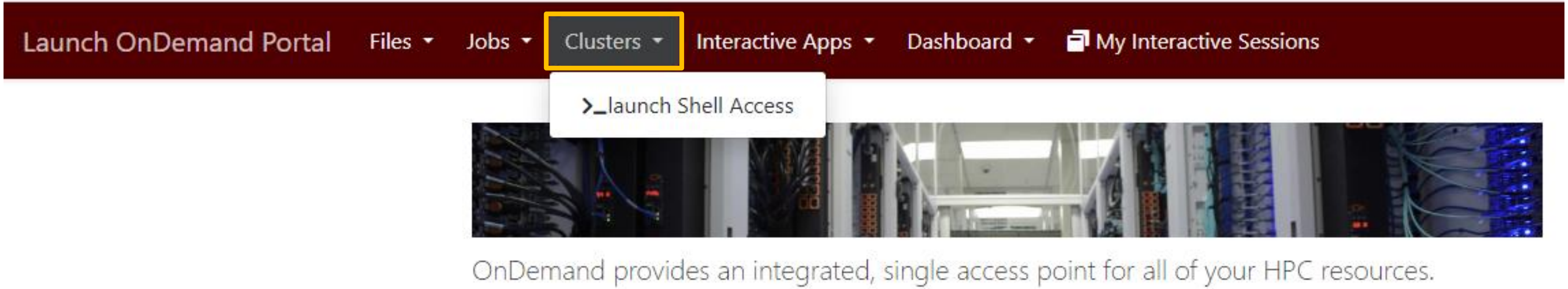


Select the Identity Provider appropriate for your account.

It uses two-factor authentication with DUO.

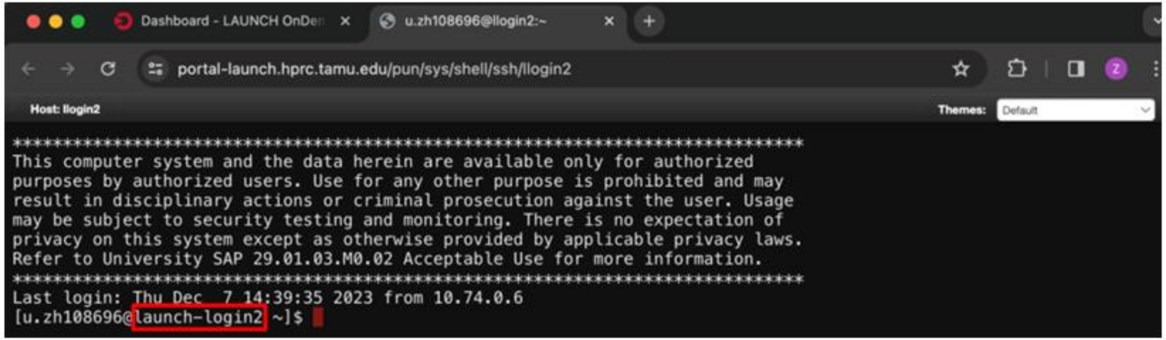
Shell access via the HPRC Portal

–Top Banner Menu “Clusters” -> “Shell Access”



Success!

- Welcome to the Launch login node!
- Check which login node you are on.



Access A30 GPUs

Check the A30 GPUs by using the command below.
$ pestat -p gpu –G
Or $ pestat -p gpu -G | grep a30

```
[u.lz104503@launch-login2 ~]$ pestat -p gpu -G
Print only nodes in partition gpu
GPU GRES (Generic Resource) is printed after each JobID
Hostname       Partition     Node Num_CPU  CPUload  Memsize  Freemem  GRES/node         Joblist
                            State Use/Tot  (15min)     (MB)     (MB)                     JobID User GRES/job ...
lg01                 gpu    idle     0 192    0.00   760000   698405  gpu:a30:2(S:0-1)
lg02                 gpu    idle     0 192    0.00   760000   756214  gpu:a30:2(S:0-1)
lg03                 gpu    idle     0 192    0.00   760000   764855  gpu:a30:2(S:0-1)
lg04                 gpu    idle     0 192    0.00   760000   767164  gpu:a30:2(S:0-1)
lg05                 gpu    idle     0 192    0.00   760000   765299  gpu:a30:2(S:0-1)
lg06                 gpu    idle     0 192    0.00   760000   768472  gpu:a30:2(S:0-1)
lg07                 gpu    idle     0 192    0.00   760000   766180  gpu:a30:2(S:0-1)
lg08                 gpu    idle     0 192    0.00   760000   767975  gpu:a30:2(S:0-1)
lg09                 gpu    idle     0 192    0.00   760000   768860  gpu:a30:2(S:0-1)
lg10                 gpu    idle     0 192    0.00   760000   768878  gpu:a30:2(S:0-1)
```

Lab 2: Run Two AI models on A30 GPUs

Commands to copy the materials

- Navigate to your personal scratch directory

  $ cd $SCRATCH
- Clone the repository into your personal scratch directory

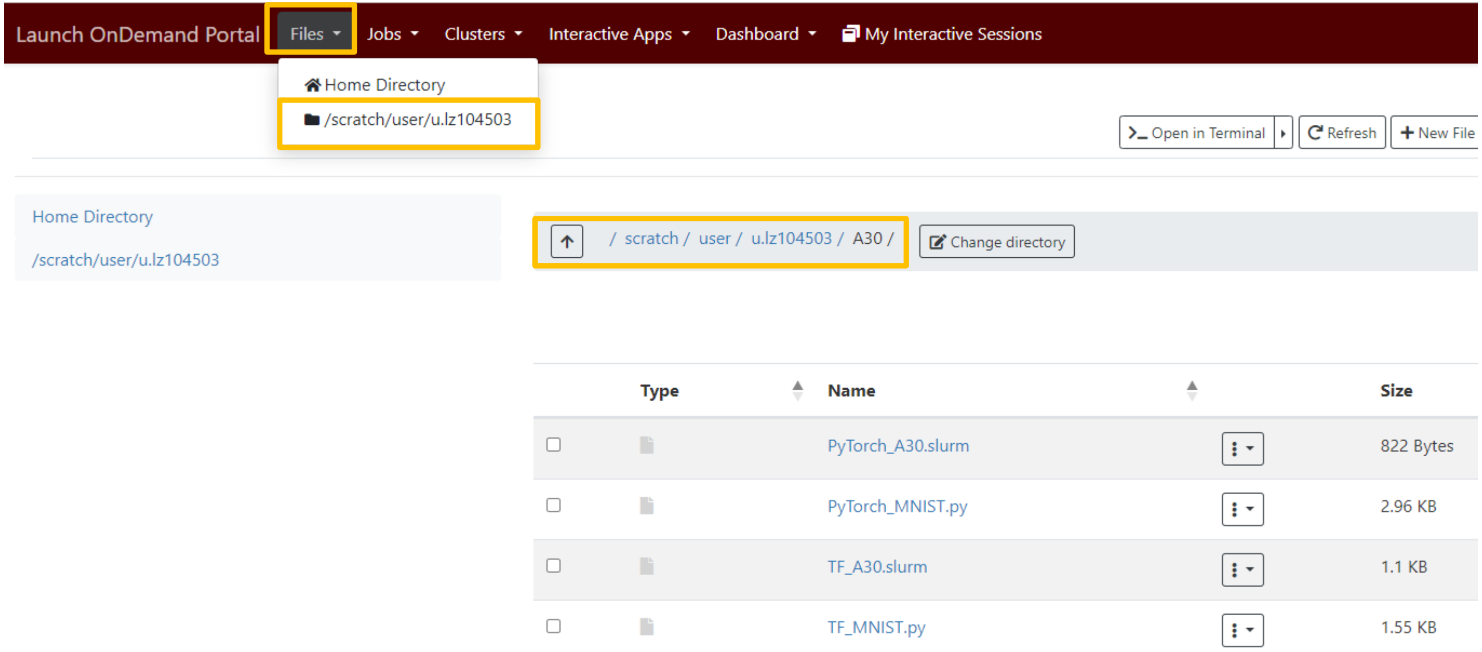
  $ git clone https://github.com/zhailujun123/A30.git
- Enter this directory

  $ cd A30
- 4 files are listed

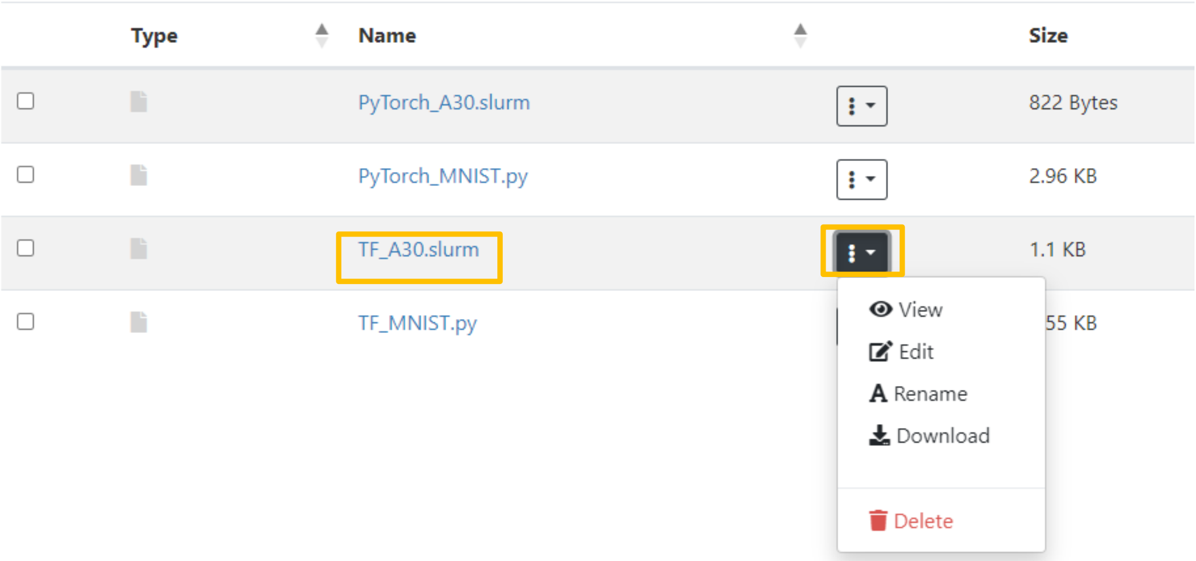
  $ ls

```
PyTorch_A30.slurm  PyTorch_MNIST.py  TF_A30.slurm  TF_MNIST.py
```

Option 2


Launch OnDemand Portal
Files
Jobs
Clusters
Interactive Apps
Dashboard
My Interactive Sessions
Home Directory
/scratch/user/u.lz104503
Open in Terminal
Refresh
New File
Home Directory
/scratch/user/u.lz104503
/ scratch / user / u.lz104503 / A30 /
Change directory
Type
Name
Size
PyTorch_A30.slurm
822 Bytes
PyTorch_MNIST.py
2.96 KB
TF_A30.slurm
1.1 KB
TF_MNIST.py
1.55 KB

Introduction to Slurm file



Introduction to Slurm file

```
#!/bin/bash
##NECESSARY JOB SPECIFICATIONS

#SBATCH --job-name=TF_MNIST
#SBATCH --time=01:00:00
#SBATCH --nodes=1
#SBATCH --output=TF_MNIST.slurm_run.%j
#SBATCH --mem=480GB
#SBATCH --partition=gpu
#SBATCH --gres=gpu:a30:1
##SBATCH --nodelist=lg01
##SBATCH --exclusive
##SBATCH --reservation=training


# Load required modules in the correct order
module load WebProxy
#module load GCCcore/11.3.0  Python/3.10.8
module load GCCcore/12.2.0  Python/3.10.8
module load GCC/12.2.0 OpenMPI/4.1.4 CUDA/11.7.0 TensorFlow/2.13.0



# Verify module loading
module list

# Check Python version
#python_version=$(python --version 2>&1)
#echo "Python Version: $python_version"


# Activate the Python virtual environment
python -m venv cvenv # Create virtual env
source cvenv/bin/activate



# Upgrade pip and resolve TensorFlow dependencies
pip install --upgrade pip
pip install wheel==0.36.2 six==1.16.0 requests==2.25.1 packaging==20.9 typing-extensions==3.7.4.3 urllib3==1.26.5



cd $SCRATCH/A30
python TF_MNIST.py
```

Specifies the maximum runtime for the job.

Multiple nodes can be requested

Job submission—Run a Tensorflow model on A30

- Use Sbatch command to submit a job

  $ sbatch TF_A30.slurm

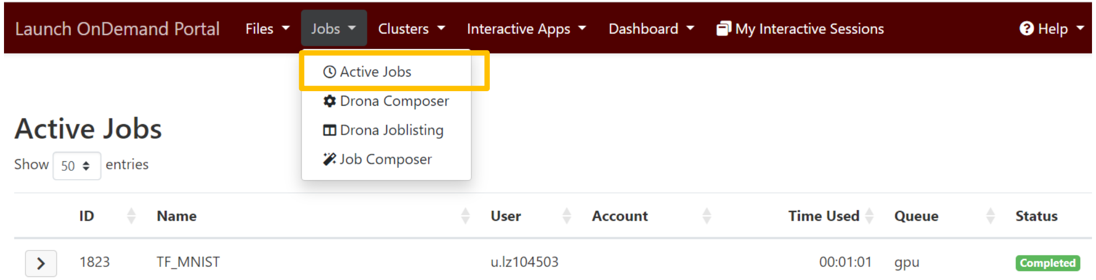
```
[u.lz104503@launch-login2 A30]$ sbatch TF_A30.slurm
Submitted batch job 1819
[u.lz104503@launch-login2 A30]$
```

Job status-Two ways to check

- Check job status

  $ pestat -p gpu -G

```
[u.lz104503@launch-login2 A30]$ pestat -p gpu -G
Print only nodes in partition gpu
GPU GRES (Generic Resource) is printed after each JobID
Hostname       Partition     Node Num_CPU  CPUload  Memsize  Freemem  GRES/node        Joblist
                            State Use/Tot  (15min)     (MB)     (MB)                   JobID User GRES/job ...
lg01                 gpu      mix   1 192    0.11*   760000   698373  gpu:a30:2(S:0-1) 1824 u.lz104503 gpu:a30=1
lg02                 gpu     idle   0 192    0.06    760000   756201  gpu:a30:2(S:0-1)
lg03                 gpu     idle   0 192    0.11    760000   764508  gpu:a30:2(S:0-1)
lg04                 gpu     idle   0 192    0.00    760000   767161  gpu:a30:2(S:0-1)
lg05                 gpu     idle   0 192    0.00    760000   765299  gpu:a30:2(S:0-1)
lg06                 gpu     idle   0 192    0.00    760000   768471  gpu:a30:2(S:0-1)
lg07                 gpu     idle   0 192    0.00    760000   766179  gpu:a30:2(S:0-1)
lg08                 gpu     idle   0 192    0.00    760000   767974  gpu:a30:2(S:0-1)
lg09                 gpu     idle   0 192    0.00    760000   768860  gpu:a30:2(S:0-1)
lg10                 gpu     idle   0 192    0.00    760000   768877  gpu:a30:2(S:0-1)
```



Job submission—Run a Pytorch model on A30

- Use Sbatch command to submit a job

  $ sbatch PyTorch_A30.slurm

Thank you !!

Vim - a text editor

```
vim your_file.py
```

1. `vim your_file.py`
   - This command opens the file named `your_file.py` in the Vim text editor. If the file does not exist, Vim will create a new file with that name.
2. `I`
   - This command is used in normal mode in Vim to switch to insert mode before the cursor. When you press `I`, Vim will allow you to start typing text at the current cursor position.
3. `ESC`
   - The `Esc` key is used to exit from insert mode or any other mode and return to normal mode. It's a crucial key in Vim for transitioning between different modes.
4. `:w`, `:wq`, `:q!`
   - These are command-line commands you can use while in normal mode or command-line mode in Vim.
     - `:w`: Writes (saves) the changes to the file without quitting Vim.
     - `:wq`: Writes the changes to the file and quits Vim.
     - `:q!`: Quits Vim without saving changes, discarding any modifications made.